ErasmusBuddy Project Documentation

Team Members:

- Baver Arslanargun (Authentication, Firebase setup)
- Poyraz Erdoğan (TravelIdeaService, detail screen)
- Berhan Kemal (Travel idea list)
- Natoli Hunduma Legesse (Add travel idea form)
- Umut Tuncer (Home screen, README/docs)

Development Workflow

ErasmusBuddy was developed using a structured Git branching strategy across a five-person team.

All development work was carried out on isolated feature branches created from the `development` branch. Each feature or fix was submitted as a Pull Request targeting `development`, reviewed, and merged only after passing `flutter analyze` with no errors. This ensured that the `development` branch always remained in a stable, runnable state throughout the project.

Upon completion of all features, the `development` branch was merged into `main` as a tagged `v1.0.0` release, representing the final production-ready version of the application submitted for grading.

Branch examples:

- `feature/travel-idea-list-ui`
- `feature/list-travelidea-model`
- `feature/list-firestore-data`
- `feature/list-error-states`
- `feature/pass-idea-id-to-detail`

This workflow mirrors industry-standard practices, providing a clear and auditable commit history that reflects each team member's individual contributions.

---

1. Technical Documentation

1.1 Project Overview & System Requirements

ErasmusBuddy is a Flutter-based mobile application designed as an MVP (Minimum Viable Product) for Erasmus students to share and discover travel itineraries. Android is the primary tested and verified target platform for this Firebase-connected application.

1.2 How to Run the Application

To set up and run the application locally, follow these steps:

1. **Prerequisites:** Ensure you have the Dart SDK (compatible with `^3.10.4`), Git, and an Android Emulator (or physical device) configured.

2. **Clone the Repository:** `git clone https://github.com/baverarslanargun/erasmusbuddy.git` `cd erasmusbuddy`
3. **Install Dependencies:** Fetch the required packages specified in `pubspec.yaml`: `flutter pub get`
4. **Execute:** Launch the app on your connected Android device: `flutter run` *(Note: Firebase is pre-configured via the `google-services.json` file inside the Android directory; no additional infrastructure setup is required for local testing).*

1.3 Architectural Design

The application follows a lightweight, **Service-Oriented Layered Architecture** combined with Flutter's native **Reactive UI components**.

Directory and Layer Separation:

- **`lib/main.dart`:** The application entry point responsible for Firebase initialization, root-level route definitions, and global authentication gate management (`AuthGate`).
- **`lib/core/`:** Houses shared global configurations such as applications themes (`lib/core/theme/`), colors, and static assets.
- **`lib/models/`:** Contains lightweight Data Transfer Objects (DTOs), such as `TravelIdea`, responsible for serializing and deserializing JSON data from the database.
- **`lib/services/`:** The data isolation layer. It contains standalone service classes (`AuthService`, `TravelIdeaService`) that encapsulate backend communication.
- **`lib/screens/`:** The presentation layer composed of modular Flutter widgets that listen to data streams and render user interfaces dynamically.

1.4 Main Dependencies & Core Libraries

The system targets high efficiency with a minimal dependency footprint to avoid configuration overhead:

- **`firebase_core` (4.11.0):** Required for bootstrapping the Firebase ecosystem within the Flutter lifecycle.
- **`firebase_auth` (6.5.4):** Utilized for client-side authentication states, secure token exchanges, and user session persistence.
- **`cloud_firestore` (6.6.0):** Operates as the NoSQL document database layer for hosting and querying user travel metadata.
- **`cupertino_icons` (v1.x):** Provides asset support for interface styling.

1.5 Technical Justification of Decisions

Why Firebase (Authentication & Cloud Firestore)? Instead of developing a custom REST API with Node.js/Python and hosting a relational database (e.g., PostgreSQL), the team chose **Firebase**. For an MVP scoped project, Firebase offers out-of-the-box infrastructure, automatic connection persistence, and pre-built secure user management via Firebase Auth. **Cloud Firestore** was selected because its NoSQL document structure fits perfectly with the flexible nature of travel ideas and provides native offline caching support.

Why Native StreamBuilder/FutureBuilder over Complex State Management? The project intentionally avoids architectural frameworks like BLoC, Riverpod, or Provider. Given the MVP scope, introducing them adds unnecessary complexity and boilerplate code. Instead, we leveraged Flutter's native **`StreamBuilder`** and **`FutureBuilder`** widgets. Firestore naturally returns real-time data as `Stream<QuerySnapshot>`. By feeding

these streams directly into a `StreamBuilder`, the UI automatically re-renders whenever a new travel plan is added, achieving real-time reactive behavior with absolute architectural simplicity.

Why Android-Only Platform Scope? To maximize development velocity and ensure a flawless, bug-free implementation within the project's timeline, the team decided to restrict the Firebase configuration exclusively to the **Android platform** for this version, leaving web and desktop expansion for future release cycles.

---

2. Privacy and Accessibility

2.1 Stored Data

ErasmusBuddy uses Firebase Authentication and Cloud Firestore.

Firebase Authentication manages:

- User email addresses
- Password-based authentication (passwords are strictly handled by Firebase Authentication; the application does not store them in Firestore or local files)
- Login sessions and Firebase user IDs

Each travel idea document contains:

- Document ID, Title, Destination, Description, Duration, Budget, Creator user ID, and Creation date. *(The application does not collect GPS location, contacts, payment details, or transportation data).*

2.2 Privacy Notes

ErasmusBuddy is a student MVP. Only data needed for authentication and sharing travel ideas is collected. The current MVP does not include in-app account deletion, data export, or travel idea deletion. These limitations are acknowledged, and test data can be removed through the Firebase Console.

2.3 Accessibility

The interface uses standard Flutter Material widgets, including labeled form fields, buttons, progress indicators, and AppBar tooltips. Long forms and content screens use scrollable layouts to reduce overflow on smaller devices. Text fields include labels or hints, and validation messages explain missing input.

---

3. Design Guidelines

Use clear English and student-friendly wording. Avoid terms related to automatic route planning, transportation planning, tickets, maps, or live navigation.

Preferred terms:

- Travel idea / Travel plan idea
- Add travel idea / Explore travel ideas

Colors:

- Primary: Erasmus Blue (`#2563EB`)
- Secondary: Travel Teal (`#14B8A6`)
- Background: Soft Light (`#F8FAFC`)
- Surface: White (`#FFFFFF`)
- Main Text: Dark Slate (`#0F172A`)
- Secondary Text: Slate Gray (`#64748B`)
- Error: Soft Red (`#DC2626`)

Layout & Buttons: Keep screens simple and focused. Use enough spacing between elements. Travel ideas should be shown with small cards that include only the main details. Use primary buttons for main actions such as logging in, exploring travel ideas, or adding a travel idea.

---

4. Demo Script

Before the Demo

1. Connect the Android device to the internet.
2. Confirm that Email/Password login is enabled in Firebase Authentication.
3. Confirm that Firestore allows the required authenticated reads and writes.
4. Run the Android application.

Step 1: Register

1. Open the Login screen and tap `Create an account`.
2. Enter a username, email, password, and password confirmation.
3. Tap `Create account`. **Expected result:** Firebase creates the account and the Home screen opens.

Step 2: Logout and Login

1. Tap the Logout icon on the Home screen and confirm.
2. Enter the registered email and password, then tap `Login`. **Expected result:** The Home screen opens for the authenticated user.

Step 3: Add a Travel Idea

1. Tap `Add Travel Idea`.
2. Fill in the title, destination, duration, budget, and description.
3. Tap `Share Travel Idea`. **Expected result:** A success message appears after Firestore saves the idea, and the form fields are cleared.

Step 4: Explore Travel Ideas

1. Return to the Home screen and tap `Explore Ideas`.
2. Find the newly added travel idea in the list. **Expected result:** Firestore travel ideas appear in a scrollable list.

Step 5: View Details & Profile

1. Tap a travel idea card to review its specific details. Return to the previous screen.

2. Open `My Profile`. Review the display name, email, and personal travel idea list. **Expected result:** The selected Firestore document is displayed, and the profile correctly uses the authenticated user ID.

Step 6: Final Logout

1. Tap the Logout icon and confirm. **Expected result:** The session ends and the Login screen opens.

Backup Plan

If Firebase is temporarily unavailable, show the already installed application and explain that Authentication and Firestore require an internet connection. Do not use real personal information in demo accounts.